

Proyecto MediSync

Arquitecto de Software

Diagrama de Contexto (Nivel 1 del modelo C4): muestra el sistema MediSync y sus usuarios/sistemas externos.

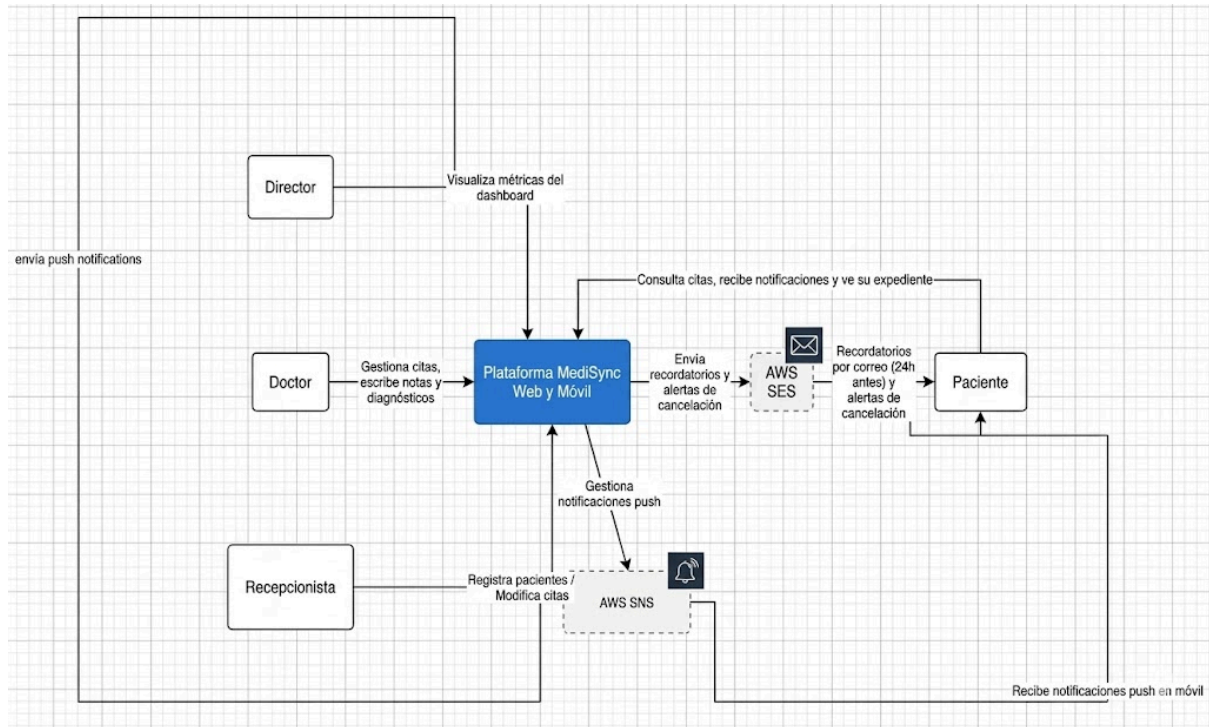
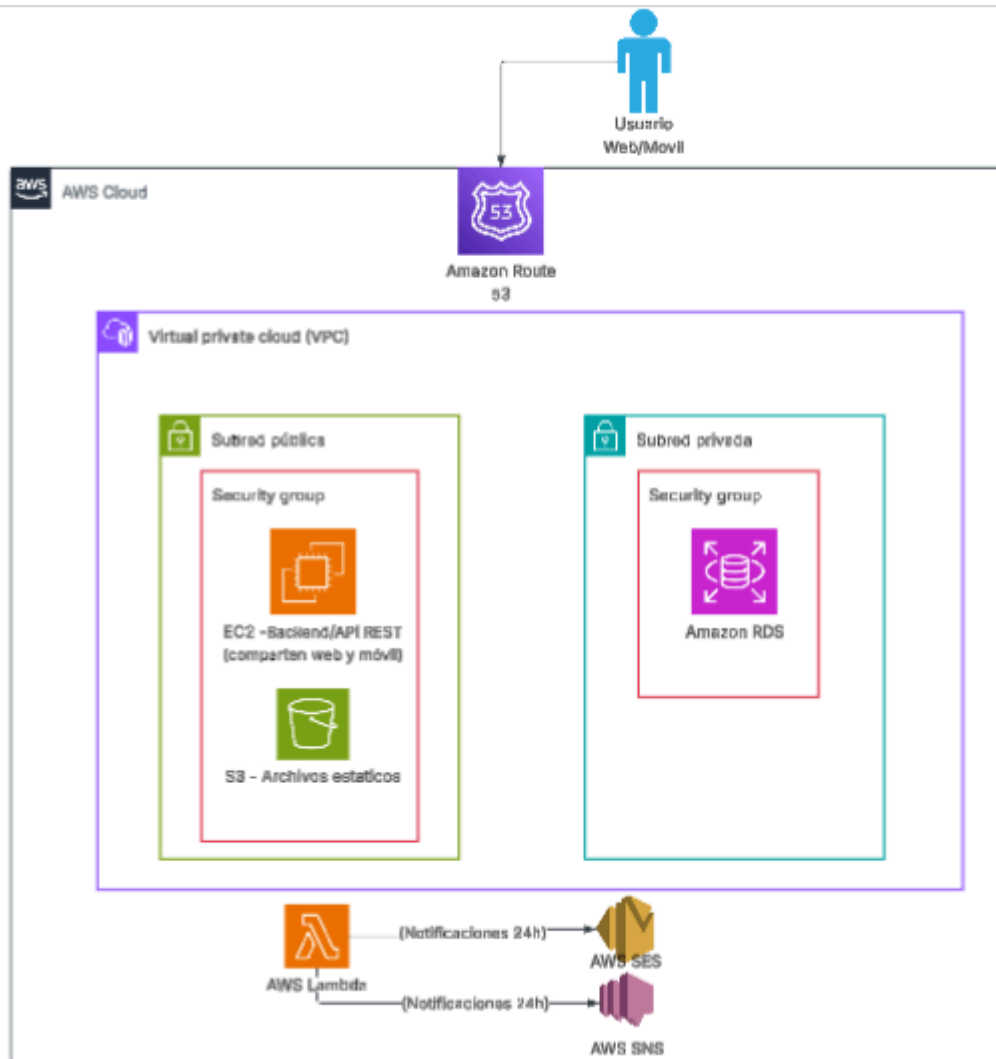


Diagrama de Contenedores (Nivel 2): muestra la separación del sistema en aplicaciones/servicios (web, móvil, API, BD, servicios AWS).



Justificación del estilo arquitectónico elegido: ¿Monolito, microservicios, serverless, event-driven? ¿Por qué? ¿Qué atributos de calidad se priorizan? ¿Qué se descartó y por qué?

Se decidió por un estilo monolito modular, básicamente consiste en que cada parte como pacientes, citas, médicos, etc. Tenga su propio módulo; ósea que tenga su propia lógica de acceso a datos, controlador y servicio con el que puedan comunicarse con otros módulos. La razón de esto es principalmente para tener un bajo acoplamiento, ya que es prioritario que la aplicación funcione en todo momento para darle el respectivo seguimiento a los pacientes, además si en algún momento se requiere que en algún módulo como "agenda de citas" necesite tener su propio servidor por la alta demanda, se pueda aislar para correrlo con un servidor más potente.

Se descartó el monolito normal por su alto acoplamiento en el código, a diferencia del modular que nos permite aislar la lógica de cada parte y es más fácil darle mantenimiento, microservicios a pesar de ser buena opción por su bajo acoplamiento es algo que en este momento no es necesario, ya que no existe como tal una alta demanda, además de su alto costo por su arquitectura como tal. Serverless no es conveniente por que se requiere que tanto las notificaciones como la agenda estén siempre disponibles ya que no son funciones aisladas, y event-driven debido a su arquitectura se tendría que configurar un intermediario para comunicarse con los demás módulos lo cual añadiría complejidad, y por otro lado sería ineficiente si es que el módulo receptor se encuentra ocupado, ya que luego no habría datos consistentes, a diferencia del monolito que todo hace en un mismo momento.

Stack tecnológico definido para todos los componentes del sistema, con justificación.

Gestión de proyecto

Tecnología	Justificación
Jira	Gestión de sprints, historias de usuario, backlog, etc.
GitHub/Git	Control de versiones y colaboración del equipo
Discord	Canal de reuniones
Whatsapp	Canal rápido de comunicación entre el equipo
Drive	Documentos del proyecto accesibles para todos
Canvas	Entrega de avances

Infraestructura AWS

Tecnología	Justificación
EC2	Servidor principal del sistema
RDS	Base de datos para datos estructurados como pacientes, citas y expedientes
S3	Fotos de médicos, documentos del expediente médico digital
Lambda	Función puntual que se dispara 24h antes de cada cita para enviar recordatorios
Docker	Estandariza el entorno de desarrollo local y de despliegue en EC2
AWS SES	Notificaciones por correo
JWT	Autenticación y roles
AWS Route 53	Dominio personalizado
AWS SNS	Notificaciones por celular
Terraform	Permite definir y versionar toda la infraestructura AWS como código

API

Tecnología	Justificación
API REST	Para comunicación entre frontend y backend
POSTMAN	Probar y documentar endpoints durante el desarrollo

Dev Web/Móvil

Tecnología	Justificación
React	Arquitectura basada en componentes reutilizables
React Native	Comparte lenguaje y lógica con React

	web
Javascript	Se usa tanto en frontend como en backend
CSS	Tecnología estándar para el diseño
Node.js	Permite usar JavaScript en el backend
Express	Facilita el manejo de rutas, peticiones y respuestas sobre Node.js
Claude	Asistente de IA utilizado como apoyo técnico durante el desarrollo.
Kiro	Permite generar código, scaffolding y configuración de infraestructura

Boceto de la especificación de la API (endpoints principales, verbos HTTP, modelos de datos).

Autenticación

Método	Endpoint	Descripción
POST	/api/auth/login	Iniciar sesión
POST	/api/auth/logout	Cerrar sesión

Gestión de pacientes

Método	Endpoint	Descripción
GET	/api/pacientes	Listar todos los pacientes
GET	/api/pacientes/{id}	Ver detalle de un paciente
POST	/api/pacientes	Registrar nuevo paciente
PUT	/api/pacientes/{id}	Actualizar datos del paciente
DELETE	/api/pacientes/{id}	Eliminar paciente

Agenda de citas

Método	Endpoint	Descripción
GET	/api/citas	Listar todas las citas

GET	/api/citas/{id}	Ver detalle de una cita
GET	/api/citas/medico/{id}	Ver citas por médico
GET	/api/citas/paciente/{id}	Ver citas por paciente
GET	/api/citas/dia/{fecha}	Ver citas por día
POST	/api/citas	Agendar nueva cita
PUT	/api/citas/{id}	Modificar cita
DELETE	/api/citas/{id}	Eliminar cita

Expediente médico digital

Método	Endpoint	Descripción
GET	/api/expedientes/paciente/{id}	Ver expediente del paciente
POST	/api/expedientes	Crear nota de consulta
PUT	/api/expedientes/{id}	Actualizar nota de consulta

Catalogo de médicos

Método	Endpoint	Descripción
GET	/api/medicos	Listar todos los médicos
GET	/api/medicos/{id}	Ver perfil de un médico
GET	/api/medicos/{id}/horarios	Ver horarios disponibles del médico
POST	/api/medicos	Registrar nuevo médico
PUT	/api/medicos/{id}	Actualizar perfil del médico
DELETE	/api/medicos/{id}	Eliminar médico

Dashboard administrativo

Método	Endpoint	Descripción
GET	/api/dashboard/pacientes-atendidos	Total de pacientes atendidos
GET	/api/dashboard/citas-por-medico	Citas agrupadas por médico
GET	/api/dashboard/ingresos	Ingresos estimados
POST	/api/dashboard/tasa-cancelacion	Tasa de cancelación de citas

Cambios: